
Computer lab exercises 1

TANA21/22: BERÄKNINGSMATEMATIK

Andrew R. Winters

HT1 2026

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Name: _____ Group: _____

LiU ID: _____

E-mail: _____

Approved: _____ Signature: _____

Corrections: _____

The goal of this set of computer exercises is to explore:

1. The numerical errors present in floating point computations.
2. Understanding errors and what they tell us about the results.
3. Fixed point iterations to solve a nonlinear equation.
4. Conditioning of linear systems and how it affects the numerical solution.

1.1 Finite precision arithmetic

This exercise will highlight some important nuances of working in a floating point number system. In Python all floats are treated in *double precision* by default.

- (a) Compute the value of the expression

$$\left| 3 \left(\frac{4}{3} - 1 \right) - 1 \right|$$

Answer: _____

- (b) Why was this value not zero? How does its magnitude compare to unit roundoff $\mu \approx 1.11 \times 10^{-16}$ (also called *machine precision*)? *Tips:* Think about representing the fraction $1/3$ as a floating point number.

- (c) We know that it is not defined to divide by zero. What happens if we try to do this on a computer? To find out evaluate the expression

$$\frac{1 - \cos(x)}{x^2} \quad \text{at} \quad x = 0$$

Tips: Remember to use the cosine from **numpy**.

Answer: _____

- (d) Use pen-and-paper and the Taylor expansion $\cos(x) = 1 - \frac{x^2}{2} + \frac{x^4}{24} + \mathcal{O}(x^6)$ to compute the limit

$$\lim_{x \rightarrow 0} \frac{1 - \cos(x)}{x^2}$$

Answer: _____

How does this value compare to the one from (c)?

1.2 Absolute versus relative error

Use a `for` loop to compute the absolute and relative errors to explore the accuracy of Stirling's approximation of the factorial

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n, \quad e = \exp(1)$$

for $n = 7, \dots, 12$. Python has a built-in `factorial(n)` function in its math library (`from math import factorial`) you can use to compute the exact values. **OBS!** The errors for this task between an exact value x_{ex} and an approximate value x_{app} are computed as

$$\text{Absolute error: } |x_{\text{ex}} - x_{\text{app}}| \quad \text{Relative error: } \frac{|x_{\text{ex}} - x_{\text{app}}|}{|x_{\text{ex}}|}$$

- (a) Fill in the remainder of the following table using the same number formatting as the provided values, e.g., with `print(f'{x:.4e}')`. See Python's docs for details on string formatting.

n	$n!$	Stirling's approximation	Absolute error	Relative error
1	1	9.2214e-01	7.7863e-02	7.7863e-02
2	2	1.9190e+00	8.0996e-02	4.0498e-02
3	6	5.8362e+00	1.6379e-01	2.7298e-02
4	24	2.3506e+01	4.9382e-01	2.0576e-02
5	120	1.1802e+02	1.9808e+00	1.6507e-02
6	720	7.1008e+02	9.9218e+00	1.3780e-02
7				
8				
9				
10				
11				
12				

- (b) Discuss the behavior of the absolute error and the relative error as the value of n increases?

- (c) Which error type is more appropriate to describe the accuracy of Stirling's approximation? Why?

1.3 Fixed point iteration

This task explores the use of the Newton-Raphson and Steffensen's fixed point methods to approximate the root x^* of the equation

$$f(x) = 3x - \cos(2\pi x) = 0,$$

in the interval $[-\frac{1}{2}, \frac{1}{2}]$. In particular, we will explore how the error evolves in each step of the iteration.

For this function $f(x)$ we know, from trigonometry, that the exact value of the root is

$$x^* = \frac{1}{6}$$

Knowing the solution is very useful because it will help us validate theoretical predictions of the Newton-Raphson and Steffensen methods. It was discussed in the lecture that the error of a quadratically convergent method, comparing the error in one iteration to the next, behaves as

$$\frac{|x^* - x_{k+1}|}{|x^* - x_k|^2} = C(x_k),$$

where the value of C might depend on the current approximation to the root. In practice, the value of C is unknown. However, it is possible to create a **rough** estimate of the error by assuming that $C(x_k) \approx 1$, i.e.,

$$|x^* - x_{k+1}| \approx |x^* - x_k|^2.$$

Taking the logarithm of both sides and manipulating gives that

$$\log(|x^* - x_{k+1}|) \approx \log(|x^* - x_k|^2) \Rightarrow \frac{\log(|x^* - x_{k+1}|)}{\log(|x^* - x_k|)} \approx 2.$$

That is, comparing a ratio of the logarithm of subsequent error values should be approximately equal to the convergence rate of 2.

We use this estimate to compute the *experimental order of convergence* (EOC). This allows us to validate an algorithm (or method) because we can check if the behavior of the implementation matches what we expect from the mathematical theory.

- (a) The derivative of the function $f(x)$ needed by Newton-Raphson is $f'(x) = 3 + 2\pi \sin(2\pi x)$. Create *lambda functions* in Python for f and f' with the commands:

```
f = lambda x: 3 * x - cos(2 * np.pi * x)
fprime = lambda x: 3 + 2 * np.pi * sin(2 * np.pi * x)
```

OBS! Lambda functions are very useful to define and pass functions in Python.

- (b) Download the Newton-Raphson method `newtonRaphson.py` from the Course documents. Run the Newton-Raphson method for the problem above using the command

```
from newtonRaphson import newtonRaphson
xVals, iter = newtonRaphson(f, fprime, x0, tol)
```

with initial guess $x_0 = 0.4$ and tolerance `tol = 1e-9`. Executing `newtonRaphson` as above outputs a one dimensional array `xVals` containing the approximate root produced by the Newton-Raphson method in each iteration as well as `iter`, the number of iterations required to reach the error tolerance.

How many iterations does it take for the Newton-Raphson to compute the root to the defined tolerance?

Answer: _____

- (c) Analyze this solution output data stored in `xVals`. Fill in the table below reporting the approximate root, the function evaluation, the error in each iteration and compute the experimental order of convergence. You can evaluate the lambda function at all the data with `f(xVals)`.

Remember the exact root is $x^* = 1/6$.

Note, depending how quickly Newton-Raphson converges this table may not be completely full of values.

k	x_k	$f(x_k)$	$ x^* - x_k $	$\frac{\log(x^* - x_k)}{\log(x^* - x_{k-1})}$ or EOC
1	4.0000e-01	2.0090e+00	2.3333e-01	—

Compute the average EOC over all the Newton-Raphson iterations.

Answer: _____

- (d) Discuss the experimental order of convergence values in your completed table. Is the value exactly two? If not, why? How does the EOC behave as the approximation gets closer to the root?

- (e) Steffensen's method is also quadratically convergent for root finding problems

$$x_{k+1} = x_k - \frac{(f(x_k))^2}{f(x_k + f(x_k)) - f(x_k)}$$

Modify the routine in `newtonRaphson.py` to perform Steffensen's method instead.

OBS! Steffensen's method no longer requires the lambda function `fprime`.

Run your implementation of Steffensen's method on the same root finding problem with initial guess $x_1 = 0.4$ and error tolerance `tol = 1e-9`. How many iterations were required for convergence?

Answer: _____

- (f) Similar to your analysis in (c), experimentally verify for this problem that Steffensen's method is quadratic (and that your implementation is working properly). Fill in the table below with the approximate root, the function evaluation, the error in each iteration, and the EOC.

k	x_k	$f(x_k)$	$ x^* - x_k $	$\frac{\log(x^* - x_k)}{\log(x^* - x_{k-1})}$ or EOC
1	4.0000e-01	2.0090e+00	2.3333e-01	—

Compute the average EOC over all iterations of Steffensen's method.

Answer: _____

- (g) Compare the results, the iteration count, the EOC, etc., produced by the Newton-Raphson and Steffensen methods. Draw conclusions as to *one advantage* and *one disadvantage* of both methods.

Newton-Raphson

Advantage:

Disadvantage:

Steffensen

Advantage:

Disadvantage:

1.4 Conditioning of linear systems

Next, we explore the sensitivity and reliability of solving a linear system $\mathbf{Ax} = \mathbf{b}$ on a computer. Also, it will demonstrate the rather strange behavior that solving a problem in floating point arithmetic can exhibit.

- (a) Consider the 2×2 linear system

$$\mathbf{Ax} = \begin{pmatrix} 0.913 & 0.659 \\ 0.457 & 0.330 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0.254 \\ 0.127 \end{pmatrix} = \mathbf{b}.$$

Input this matrix and right-hand-side with the commands

```
A = np.array([[0.913, 0.659], [0.457, 0.33]])
b = np.array([0.254, 0.127])
```

Verify that the exact solution is $\mathbf{x} = (1, -1)^T$ with commands:

```
x_exact = np.array([1, -1])
print(A @ x_exact - b)
```

Use the in-built functionality from `numpy` to solve the system numerically with

```
x_num = np.linalg.solve(A, b)
```

Write your answer with all 16 digits from double precision (set `np.set_printoptions(precision=16, suppress=False)` before printing any output)

Answer: _____

How is the solution `x_num` different from the true solution `x_exact`?

- (b) Examine how perturbations in the right-hand-side can affect the numerical solution of this problem. To do so, slightly perturb the right-hand-side in two ways to have two new systems

$$(i) \quad \begin{pmatrix} 0.913 & 0.659 \\ 0.457 & 0.330 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0.2539949 \\ 0.1272061 \end{pmatrix} = \hat{\mathbf{b}}, \quad (ii) \quad \begin{pmatrix} 0.913 & 0.659 \\ 0.457 & 0.330 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0.252428 \\ 0.126213 \end{pmatrix} = \tilde{\mathbf{b}}.$$

Input these new right-hand-side vectors into Python with the commands

```
b_hat = np.array([0.2539949, 0.1272061])
b_tilde = np.array([0.252428, 0.126213])
```

Use the solve command, t.ex. `x_hat = np.linalg.solve(A, b_hat)`, and compute the solution to each of the perturbed problems. Give your answers in double precision.

System (i) answer `x_hat`: _____

System (ii) answer `x_tilde`: _____

- (c) Compute the vector 1-norm of the residual $\|\mathbf{r}\|_1 = \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_1$ for the perturbed solutions `x_hat` and `x_tilde`.

OBS! Compute the residual with the original right-hand-side `b` from Part (a). That is,

$$\hat{\mathbf{r}} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} \quad \text{and} \quad \tilde{\mathbf{r}} = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}}$$

Use `numpy` to compute the 1-norms, e.g., `np.linalg.norm(r_hat, ord=1)`. From these residual computations, which seems most accurate, `x_hat` or `x_tilde`? Why?

- (d) Directly compare the exact solution `x_exact` from Part (a) given to the perturbed solutions `x_hat` and `x_tilde` from Part (b). Which of the perturbed solutions is most accurate? Why?

- (e) We will use the condition number of the matrix `A` to partially explain this rather unpredictable behavior in the numerical solution of this 2×2 system. The solution to a linear system can be bounded by

$$\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \frac{\|\Delta \mathbf{b}\|}{\|\mathbf{b}\|}$$

where the errors are measured in a vector norm and $\kappa(\mathbf{A})$ is the condition number of the matrix `A`.

The condition number acts like an *amplification factor* that bounds the relative change in the solution `x` due to a given change in the right-hand-side data `b`. Qualitatively, the *matrix condition number* measures how “sensitive” the linear system is to small changes (also called perturbations).

If the condition number is close to one (well-conditioned) the solution **is not** sensitive to perturbations in the right-hand-side. Alternatively, if the condition number is large (ill-conditioned) the solution **is very** sensitive to changes in the right-hand-side.

- Use the `numpy` command `np.linalg.cond(A, p=1)` to compute the condition number $\kappa_1(\mathbf{A})$ of the matrix `A` in the 1-norm.

Answer: _____

From $\kappa_1(\mathbf{A})$ and the results in Parts (b)-(d) is the matrix `A` ill-conditioned or well-conditioned? Why?